

Node.js Interview Questions and Answers



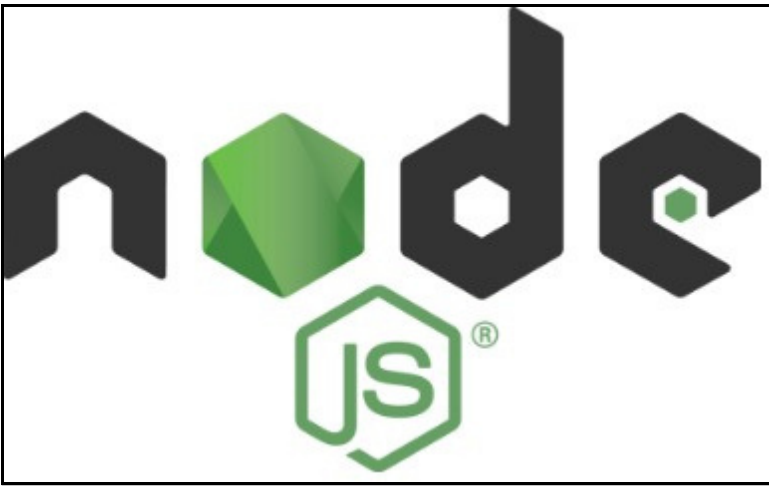
NodeJS Interview Questions for Fresher & Senior Developer

Node.js Interview Questions and Answers: An Overview

Node.js is a server-side JavaScript environment for developing web applications like ASP.NET, JSP, Php, etc. It is an open-source and cross-platform framework based on Google's V8 JavaScript Engine. In this [Node.js Tutorial](#), we have tried our best to provide you with some of the top Node.Js Interview Questions and Answers.

1. What is Node.js?

Node.js is a server-side JavaScript environment for developing web applications like [ASP.NET](#), JSP, PHP, etc. It is an open-source and cross-platform framework based on Google's V8 JavaScript Engine.



It is used to build fast and scalable network applications as well as data-intensive real-time web applications. All versions of Node.js are starting from 0.1.0 releases to 0.1.x, 0.2.x, 0.3.x, 0.4.x, 0.5.x, 0.6.x, 0.7.x, 0.8.x, 0.9.x, 0.10.x, 0.11.x, and 0.12.x. Before merging of Node.js and io.js, its last version was Node.js v0.12.9.

Read More: [Brief History of node.js and io.js](#)

2. What is the Node.js foundation?

The Node.js Foundation is an independent foundation to take care of the development and release of Node.js. It has developers from IBM, Microsoft, PayPal, Joyent, Fidelity, SAP, and other companies. On Sep 14, 2015, the Node.js foundation announced the combined release of Node.js and io.js into a single code base known as Node.js version 4.0. It has a feature of Node.js and io.js including a lot of new features of ES6.

3. What is the V8 JavaScript Engine?

V8 is an open-source **JavaScript engine** developed by Google in 2008 to be used in the Chrome browser. It is written in C++ language and implements ES5.



Key Points About V8 JavaScript Engine

- It can be run standalone or can be embedded into any C++ application.
- It uses just-in-time compilation (JIT) to execute JavaScript code.
- It compiles JavaScript to native machine code (IA-32, x86-64, ARM, or MIPS ISAs) before execution.
- It is used by many open-source projects like Node.js and MongoDB to execute JavaScript on the server side.

4. What IDEs can you use for Node.js development?

Node.JS development can be done with the help of the following IDEs:

1. **Visual Studio 2013, 2015 or higher**
2. **Visual Studio Code**
3. **Atom**
4. **Node Eclipse**
5. **WebStorm**
- . **Sublime Text**

5. What platforms do Node.js support?

Node.js supports the following platforms:

1. **Linux**
2. **Windows**
3. **Mac OS X**
4. **SunOS**

6. Where can you deploy the Node.js web application?

The easiest way to deploy your Node.js web application is by using Cloud server hosting like Windows Azure, Aws, Google, Heroku, etc.

7. What is callback?

A callback is a function passed as an argument to an asynchronous function, that describes what to do after the asynchronous operation has been completed. Callbacks are used frequently in node.js development.

```
var fs = require('fs');  
//callback function to read file data  
fs.readFile('text.txt', 'utf8', function (err, data) { //callback function  
  console.log(data);  
});
```

8. What is a Module?

A module is a collection of JavaScript code that encapsulates related code into a single unit of code. Node.js has a simple module loading system. A developer can load a JavaScript library or module into his program by using the required method as given below:

```
var HTTP = require('HTTP');
```

Read More: [Exploring Node.js Core Modules](#)

9. What is a REPL Terminal?

REPL stands for Read-Eval-Print-Loop. It is an interface to run your JavaScript code and see the results. You can access REPL by simply running a node.js command prompt and simply running the command node.

```
C:\> node
Your environment has been set up for using Node.js 4.2.4 (x64) and npm.
C:\Users\Shailendra Chauhan> node
> 1+2
3
>
```

Here, we are adding two numbers 1 and 2 which results in 3.

10. What is the difference between Package dependencies and development dependencies?

Package dependencies and development dependencies, both are defined in the package.json file.

Package Dependencies

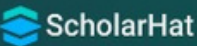
The dependencies field of the package.json file will have all packages listed on which your node project is dependent.

```
{
  "dependencies": {
    "angular": "1.4.8",
    "jQuery": "^2.1.4"
  }
}
```

To do a listing of your node module as a dependencies package you need to use either `-save` flag or `-production` flag with the node command to install the package.



Node.js Course For Beginners



- ✓ Learn Node.js From Scratch
- ✓ Covers Advanced Node.js
- ✓ Doubt Support using Discord
- ✓ Get Course Certificate

- ✓ 60+ Video Lessons
- ✓ 10+ Hands-On Labs
- ✓ 24X7 LMS Access

- ✓ 3 Projects
- ✓ Quick Notes
- ✓ Skill Tests

[LEARN MORE](#)

Development Dependencies

The devDependencies field of the package.json file will have those packages listing which is only required for testing and development.

```
"devDependencies": {  
  "mocha": "~1.8.1"  
}
```

To do a listing of your node module as a dependencies package you need to use `-dev` flag with the node command to install the package.

11. What are buffers?

JavaScript language has no mechanism for reading or manipulating streams of binary data. So, Node.js introduced the Buffer class to deal with binary data. In this way, Buffer is a Node.js special data type to work with binary data. A buffer length is specified in bytes. By default,

buffers are returned in data events by all Stream classes. Buffers are very fast and light objects as compared to strings. A buffer acts like an array of integers, but cannot be resized.

12. What are Streams?

Typically, a Stream is a mechanism for transferring data between two points. Node.js provides you streams to read data from the source or to write data to the destination. In Node.js, Streams can be readable, writable, or both and all streams are instances of the EventEmitter class.

```
var http = require('http');  
  
var server = http.createServer(function (req, res) {  
  // here, req is a readable stream //  
  here, res is a writable stream  
});
```

13. How to debug the code in Node.js?

Static languages like C# and Java have tools like Visual Studio and Eclipse respectively for debugging. Node.js is based on JavaScript and in order to debug your JavaScript code we have `console.log()` and `alert()` statements. But Node.js supports other options as well for code debugging. It supports the following options:

- **The Built-In Debugger:** A non-GUI tool to debug the Node.js code.
- **Node Inspector:** A GUI tool to debug the Node.js code with the help of Chrome or Opera browser.
- **IDE Debuggers:** IDE like WebStorm, Visual Studio Code, Eclipse IDE, etc., support the Node.js code debugging environment.

14. What are the uses of the path module in Node.js?

Node.js provides a path module to normalize, join, and resolve file system paths. It is also used to find relative paths, extract components from paths, and determine the existence of paths.

Note - The path module simply manipulates strings and does not interact with the file system to validate the paths.

15. What is File System module in Node.js?

Node.js provides a file system module (`fs`) to perform file I/O and directory I/O operations. The `fs module` provides both asynchronous or synchronous ways to perform file I/O and directory I/O operations. The synchronous functions have “Sync” word as a suffix with their names and return the value directly.

In synchronous file I/O operation, Node doesn't execute any other code while the I/O is being performed. By default, `fs` module functions are asynchronous, which means they return the output of the I/O operation as a parameter to a callback function.

16. Which types of network applications you can build using node.js?

Node.js is best for developing the HTTP-based application. But it is not only for developing the HTTP-based application. It can be used to develop other types of applications. Like as:

- **TCP server**
- **Command-line program**
- **Real-time web application**
- **Email server**
- **File server**
- **Proxy server**

17. What are Node.js Http module limitations?

Node.js [HTTP module](#) has the following limitations:

- **No cookies handling or parsing**
- **No built-in session support**
- **No built-in routing supports**
- **No static file serving**

18. What is socket.io?

Socket.io is the most popular node.js module for WebSockets programming. It is used for two-way communication on the web. It uses events for transmitting and receiving messages between client and server.



socket.emit("eventname",data) event is used for sending messages.
socket.on("eventname",callback) event is used for receiving messages.

19. What are various node.js web development frameworks?

The best and most powerful node.js web development frameworks to build real-time and scalable web applications with ease are given below:

MVC frameworks

- Express
- Koa
- Hapi
- Sails
- Nodal

Full-stack frameworks

- Meteor
- Derby.js
- MEAN.IO
- MEAN.js
- Keystone
- Horizon

20. What are various node.js REST API frameworks?

The best and most powerful node.js REST API frameworks to build a fast Node.js REST API server with ease are given below:

- **Restify**

- **LoopBack**
- **ActionHero**
- **Fortune.js**

21. What are some differences between Angular JS and Node.js?

AngularJS	Node.js
Written in TypeScript	Written in a variety of languages, like C , C++ , and JavaScript
Great for creating highly interactive web pages	Suited for small-scale projects and applications
Open-source framework for web application development	Runtime environment based on multiple platforms
Used to create single-page applications for client-side	Used to create server-side networking applications
Helps split an application into model-view-controller (MVC) components	Helps generate queries for databases
Appropriate for developing real-time applications	Appropriate for situations requiring quick action and scaling
Angular itself is a web application framework	Node.js has many frameworks, including Express.js , Partial.js , and more

22. What is NPM?

NPM stands for Node Package Manager, responsible for managing all the packages and modules for Node.js.

Node Package Manager provides two main functionalities:

1. Provides online repositories for node.js packages/modules, which are searchable on search.npmjs.org

2. Provides command-line utility to install Node.js packages and also manages Node.js versions and dependencies

23. Why is Node.js preferred over other backend technologies like Java and PHP?

Some of the reasons why Node.js is preferred are as follows:

- Node.js is very fast
- Node Package Manager has over 50,000 bundles available at the developer's disposal.
- Perfect for data-intensive, real-time web applications, as Node.js never waits for an API to return data.
- Better synchronization of code between server and client due to the same code base
- Easy for web developers to start using Node.js in their projects as it is a JavaScript library

24. Explain the control flow function.

The control flow function is the sequence in which statements or functions are executed. Since I/O operations are non-blocking in Node.js, control flow cannot be linear. Therefore, it registers a callback to the event loop and passes the control back to the node, so that the next lines of code can run without interruption.

Example

```
[code language="javascript"]

fs.readFile('/root/text.txt', func(err, data){

  console.log(data);

});

console.log("This is displayed first");

[/code]
```

In this, the `readFile` operation will take some time; however, the next `console.log` is not blocked. Once the operation completes, you'll see the displayed data.

25. Explain Node.js web application architecture.

A web application is distinguished into four layers:

1. **Client Layer:** The Client layer contains web browsers, mobile browsers, or applications that can make an HTTP request to the web server.
2. **Server Layer:** The Server layer contains the Web server which can intercept the requests made by clients and pass them the response.
3. **Business Layer:** The business layer contains an application server which is utilized by the web server to do required processing. This layer interacts with the data layer via a database or some external programs.
4. **Data Layer:** The Data layer contains databases or any source of data.

Read More: [Exploring Node.js Architecture](#)

26. Why is assert used in Node.js?

Assert is used to explicitly write test cases to verify the working of a piece of code. The following code snippet denotes the usage of assert:

```
var assert = require('assert');
function add(x, y) {
  return x + y;
}
var result = add(3,5);
assert( result === 8, 'three summed with five is eight');
```

27. What is clustering in Node.js and how does it work?

Clustering is a technique in Node.js that allows you to create a cluster of worker processes that can share a single port and handle incoming requests in parallel. This can help improve the

performance and scalability of your Node.js applications.

In a typical Node.js application, there is a single process that handles all incoming requests. This process runs on a single CPU core and can become a bottleneck as the number of requests increases. With clustering, you can create multiple worker processes that can handle requests in parallel, spreading the load across multiple CPU cores.

Clustering works by using the built-in cluster module in Node.js. This module allows you to create a master process that manages a cluster of worker processes. The master process listens for incoming requests and distributes them to the worker processes in a round-robin fashion. Each worker process runs a copy of your application code and handles incoming requests independently.

Example



```
const cluster = require('cluster');

const http = require('http');

const numCPUs = require('os').cpus().length;

if (cluster.isMaster) {

  console.log(`Master ${process.pid} is running`);

  // Fork worker processes

  for (let i = 0; i < numCPUs; i++) {

    cluster.fork();

  }

  cluster.on('exit', (worker, code, signal) => {

    console.log(`Worker ${worker.process.pid} died`);

  });

}
```

```

    // Replace the dead worker

    cluster.fork();

  });

} else {

  // Worker process runs the server

  http.createServer((req, res) => {

    res.writeHead(200);

    res.end('Hello, world!');

  }).listen(8000);

  console.log(`Worker ${process.pid} started`);

}

```

Here, we create a master process that forks multiple worker processes. Each worker process runs a copy of the HTTP server, which listens for incoming requests and responds with a “Hello, world!” message. The master process manages the worker processes and replaces any that die unexpectedly.

28. Explain the steps to write an Express JS application.

To set up an ExpressJs application, you need to execute the following steps:

- Create a folder with the project name
- Create a file named package.json inside the folder
- Run the ‘npm install’ command on the command prompt to install the libraries present in the package file\

- Create a file named server.js
- Create the 'router' file inside the package consisting of a folder named index.js
- The application is created inside the package containing the index.html file

Read More: [Getting Started with ExpressJS](#)

29. What is the crypto module in Node.js? How is it used?

The crypto module in Node.js is used for cryptography, i.e., it includes a set of wrappers for the open SSL's hash, HMAC, sign, decipher, cipher, and verify functions.

Example of using a cipher for encryption

```
const crypto = require('crypto');

const cipher = crypto.createCipher('usnm', 'pwdd');

var encryptd = cipher.update('Welcome to hackr', 'utf8', 'hex');

encryptd += cipher.final('hex');

console.log(encryptd);
```

Example of deciphering to decrypt the above

```
const crypto = require('crypto');

const decipher = crypto.createDecipher('usnm', 'pwdd');

var encryptd = "";

var decryptd = decipher.update(encryptd, 'hex', 'utf8');
```

```
decryptd += decipher.final('utf8');  
  
console.log(decryptd);
```

30. Explain the security mechanism of Node.js.

The security mechanisms are:

- **Authorization codes:** Authorization codes help secure Node.js from unauthorized third parties. Anyone who wants to access Node.js goes through the GET request of the resource provider's network.
- **Certified Modules:** Certification modules are like filters that scan the libraries of Node.js to identify if any third-party code is present or not. Any hacking can be detected using certifications.
- **Curated Screening register:** This is a quality control system where all the packages (code and software) are checked to ensure their safety. This scan helps to eliminate unverified or unreliable libraries getting into your application.
- **Regular updates:** Downloading the newest version of Node.js will prevent potential hackers and attacks.

31. What tools can be used to ensure a consistent style in Node.js?

Following is a list of tools that can be used in developing code in teams, to enforce a given style guide and to catch common errors using static analysis.

- **JSLint**
- **JSHint**
- **ESLint**
- **JSCS**

32. Explain the difference between `setImmediate()` vs `setTimeout()`.

While the word immediate is slightly misleading, the callback happens only after the I/O events callbacks. When we call `setImmediate()`, `setTimeout()` is used to set a delay (in milliseconds)

for the execution of a one-time callback. If we execute:

```
setImmediate(function() {  
  
  console.log('setImmediate')  
  
})  
  
setTimeout(function() {  
  
  console.log('setTimeout')  
  
}, 0)
```

We will get the output as “setTimeout” and then “setImmediate.”

33. Write a function that takes an array of numbers as input and returns a new array containing only the even numbers.

```
function filterEvenNumbers(array) {  
  
  return array.filter(num => num % 2 === 0);  
  
}
```

34. What is Punycode?

Punycode is an encoding syntax in Node.js that helps convert the Unicode string of characters into ASCII. This is done as the hostnames can only comprehend ASCII codes and not Unicode. While it was bundled up within the default package in recent versions, you can use it in the previous version using the following code:



```
punycode = require('punycode');
```

35. What is piping in Node.js?

Piping is a technique for streaming data between two or more streams, such as reading from a file and writing to a network socket, or reading from one network connection and writing to another. This technique allows you to easily connect streams and transfer data efficiently, without having to manually manage the data transfer or buffer the data in memory.

The `pipe()` method takes a destination stream as its argument and returns the destination stream, allowing you to chain multiple `pipe()` calls together:



```
const fs = require('fs');

const http = require('http');

const server = http.createServer((req, res) => {

  const fileStream = fs.createReadStream('largefile.txt');

  fileStream.pipe(res);

});

server.listen(3000);
```

In the above code, we create an HTTP server that listens on port 3000. When a client requests the server, we create a new readable stream from a file called `largefile.txt` and pipe the data to the response stream using the `pipe()` method.

36. What is the framework that is used most often in Node.js today?

Node.js has multiple frameworks

- **Hapi.js**
- **Express.js**
- **Sails.js**
- **Meteor.js**
- **Derby.js**
- **Adonis.js**

Among these, the most used framework is Express.js for its ability to provide good scalability, flexibility, and minimalism.

37. What are streams in Node.js and how are they useful?

Streams in Node.js are a way of handling data continuously and efficiently. Streams allow you to read or write data piece by piece, rather than all at once, which can be useful for handling large files or data sets. They also allow you to process data in real-time, as it becomes available, which can be useful for network programming, data processing, and other applications.

Types of Streams in Nodejs

There are four types of streams in Node.js:

1. **Readable streams:** allow you to read data, piece by piece.
2. **Writable streams:** allow you to write data, piece by piece.
3. **Duplex streams:** can be both readable and writable.
4. **Transform streams:** can transform data as it passes through.

Example

```
const fs = require('fs');

const readStream = fs.createReadStream('largefile.txt');

readStream.on('data', (chunk) => {

  console.log(`Received ${chunk.length} bytes of data`);
```

```
});

readStream.on('end', () => {

  console.log('Finished reading file');

});

readStream.on('error', (err) => {

  console.error('Error:', err);

});
```

In the above example, we use the `fs.createReadStream()` method to create a readable stream for a large file and listen for the data, end, and error events.

38. What is a memory leak in Node.js? How do you detect and prevent it?

A memory leak is a type of bug that occurs when an application unintentionally retains memory that it no longer needs, rather than releasing it back to the system.

Reasons behind memory leak

- Unintentional retention of objects in memory
- Circular references that prevent objects from being garbage-collected
- Use of memory-intensive libraries or data structures

Ways to Detect and Prevent Memory Leak in NodeJS

- **Monitoring memory usage:** It is one of the simplest ways to detect a memory leak. You just need to monitor the memory usage of a Node.js application over time. If memory usage consistently increases over time, it may be an indication of a memory leak.
- **Profiling tools:** Node.js provides a built-in profiler that can be used to identify areas of an application that are using an excessive amount of memory. Other profiling tools, such as

Chrome DevTools or Node Inspector, can also be used to identify memory leaks. **Garbage**

- **collection tuning:** It can help reduce the likelihood of memory leaks. For example, increasing the heap size can help reduce the frequency of garbage collection and the likelihood of garbage collection pauses.
- **Code review:** Reviewing application code can help identify areas where memory leaks may occur. NodeJS best practices, such as avoiding circular references, freeing resources when they are no longer needed, and avoiding unnecessary object creation, can help reduce the likelihood of memory leaks.

39. Why is Node.js single-threaded?

Node.js works on the single-threaded model to ensure that there is support for asynchronous processing. With this, it makes it scalable and efficient for applications to provide high performance and efficiency under high amounts of load.

40. Explain the various types of API functions in Node.js.

The two types of API functions in Node.js are:

1. **Asynchronous/Non-blocking:** These requests do not wait for the server to respond. They continue to process the next request, and once the response is received, they receive the same.
2. **Synchronous/Blocking:** These are requests that block any other requests. Once the request is completed, only then is the next one taken up.

41. Explain middleware in Node.js.

Middleware is a function that receives the request and response objects. Most tasks that the middleware functions perform are:

- Execute any code
- Update or modify the request and the response objects
- Finish the request-response cycle
- Invoke the next middleware in the stack

42. Explain LTS releases of Node.js.

LTS or Long-Term Support is applied to release lines supported and maintained by the Node.js

project for an extended period. There are two types of LTS:

1. Active, which is actively maintained and upgraded
2. Maintenance line nearing the end of the line and maintained for a short period.

43. What are the main differences between Node.js and Javascript?

Node.js Cross-platform	JavaScript
runtime engine. Code can be run ^{open-source} outside the browser.	A high-level scripting language based on the concept of OOPS.
Used on server-side. No capabilities to	The code can run only in the browser.
add HTML tags. Can be run only on	Used on client-side.
Google Chrome's V8 engine.	Can add HTML tags.
	Can be run on any browser.
Written in C++ and JavaScript.	An upgraded version of ECMA script written in C++.

44. How is Node.js better than other frameworks?

Node.js is a server-side JavaScript runtime environment built on top of the V8 JavaScript engine, the same engine that powers Google Chrome. It makes Node.js very fast and efficient, as well as highly scalable.

45. Differentiate spawn() and fork() methods in Node.js

spawn()	fork()
Designed to run system commands.	A special instance of spawn() that runs a new instance of V8.
Does not execute any other code within the node process.	Can create multiple workers that run on the same Node codebase.

<code>child_process.spawn(command[, args][, options])</code> creates a new process with the given command.	A special case of <code>spawn()</code> to create child processes using <code>child_process.fork(modulePath[, args][, options])</code>
Creates a streaming interface (data buffering in binary format) between parent and child process. It is more useful for continuous	Creates a communication (messaging) channel between parent and child process. More useful for messaging. For
operations like data streaming (read/write). For example, streaming images/files from the spawn process to the parent process.	example, JSON or XML data messaging.

46. What is an event loop in Node.js?

The event loop is a core concept in Node.js that enables it to handle many concurrent connections with low overhead. It is a loop that continuously checks for new events and executes the associated callbacks when events are detected.

In Node.js, when a client sends a request to a server, the server creates an event associated with that request and adds it to the event loop. The event loop continuously checks for new events, and when an event is detected, it dispatches the associated callback function to handle the event.

47. What is the default scope in the Node.js application?

The module scope is the default scope in Node.JS.

48. Is Node.js the best platform for CPU-heavy applications?

CPU-intensive applications are not a strong suit of Node.js. The CPU-heavy operations block incoming requests and push the thread into critical situations.

49. What is an error-first callback?

Error-first callbacks are used to pass errors and data. If something goes wrong, the programmer has to check the first argument because it is always an error argument. Additional arguments are used to pass data.

```
fs.readFile(filePath, function(err, data) {  
  if (err) {  
    //handle the error  
  }  
  // use the data object  
});
```

50. Why must the express “app” and “server” be separated?

If we keep the app and server functionalities separate, the code can be divided into multiple modules, which reduces the dependency between modules. Each module will perform a single task. Finally, the separation of logic helps avoid duplicate code.

Summary

I hope the above questions and answers will help you in your Node.js Interview. All the above interview Questions have been taken from our newly released eBook Node.js Interview Questions and Answers.

This eBook has been written to make you confident in Node.js with a solid foundation. Also, this will help you to use Node.js in your real project.

Buy this eBook at a Discounted Price!

FAQs

Q1. What is a memory leak in Node.js?

A memory leak is a type of bug that occurs when an application unintentionally retains memory that it no longer needs, rather than releasing it back to the system.

Q2. What are the two types of API functions in Node.js?

1. Asynchronous/Non-blocking
2. Synchronous/Blocking

Q3. State the layers of Node.js web application architecture.

1. Client Layer
2. Server Layer
3. Business Layer
4. Data Layer

Q4. What is the V8 JavaScript Engine?

V8 is an open-source **JavaScript engine** developed by Google in 2008 to be used in the Chrome browser. It is written in C++ language and implements ES5.

Q5. What are Node.js Http module limitations?

- No cookie handling or parsing
- No built-in session support
- No built-in routing supports
- No static file serving